

Reinforcement Learning Robotics Take-home Report

Contents

Part 0: Environment and Toolchain Check	3
0.1 Software Environment	3
0.2 PPO Training Configuration	3
0.3 Independent Evaluation	4
0.4 Learning Curve	4
0.5 Rollout Video	5
0.6 Toolchain Verification Result	6
Part 1: Custom Environment NoisyGoalReacher	7
1.1 Base Environment and Implementation	7
1.2 Goal-Position Randomization	7
1.3 Observation Noise	8
1.4 Success Determination and Reward Function	8
Sparse Reward	9
Dense Reward	9
Reward Design Rationale	9
1.5 Environment Check and Unit Tests	10
Part 2: Custom PPO Implementation vs. SB3 Baseline	12
2.1 Implementation	12
2.2 Training Configuration	12
2.3 Comparison with the SB3 Baseline	13
2.4 Reproducible Command	13
Part 3: Reward and Robustness Study	15
3.1 Sparse Reward vs. Dense Reward	15
Comparative Analysis	15
3.2 HER (Hindsight Experience Replay)	16
What is HER	16
Why HER may help solve the sparse-reward problem	17
3.3 Noise Sensitivity	17
Training and Evaluation Setup	17
Results	17
Applying to a Real Robot	18
Part 4: Evaluation Protocol, Failure Modes, and Future Work	19
4.1 Evaluation Protocol	19
4.2 Failure Modes and Debugging	19
Dense-Reward Design Flaw: Missing Arrival Reward	19
Other Debugging Items	20
Compatibility Issues in Environment and Dependency Integration	22
4.3 Future Work	23

Part 0: Environment and Toolchain Check

This part uses Gymnasium's Reacher-v5 environment to verify that the RL training, evaluation, learning-curve plotting, and video-recording toolchain works correctly.

The training algorithm is Proximal Policy Optimization (PPO) as implemented in Stable-Baselines3. Training and inference use a GPU. The setup does not rely on Isaac Sim or Isaac Gym; physics simulation is based entirely on Gymnasium and MuJoCo.

0.1 Software Environment

The experiments use Conda to manage the Python environment. The main dependency versions are:

```
name: rl-robotics-takehome
channels:
  - defaults
dependencies:
  - python=3.11
  - pip
  - pip:
    - "gymnasium[mujoco]==1.3.0"
    - "stable-baselines3[extra]==2.9.0"
    - "pytest==8.4.1"
    - "imageio"
    - "imageio-ffmpeg"
```

The recommended way to create the environment uses the simplified core environment file:

```
conda env create -f environment.yml
conda activate rl-robotics-takehome
```

To reproduce the exact dependency versions used in the experiments, a full environment snapshot can also be used:

```
conda env create -f environment-lock.yml
conda activate rl-robotics-takehome
```

0.2 PPO Training Configuration

The PPO training budget is set to 500,000 environment interaction timesteps, using 4 parallel environments to collect training data.

The training random seed is set to 0. During training, an independent evaluation is performed every 10,000 timesteps, with each evaluation running 10 episodes. The training device is the GPU.

The full training command is:

```
python scripts/part0/train_sb3.py \
  --run-name ppo_reacher_seed_0 \
  --seed 0 \
  --total-timesteps 500000 \
```

```
--n-envs 4 \  
--eval-freq 10000 \  
--n-eval-episodes 10 \  
--device cuda
```

During training, the best-performing model is saved based on the periodic evaluation results. The best model checkpoint is located at:

```
checkpoints/part0/ppo_reacher_seed_0/best/best_model.zip
```

0.3 Independent Evaluation

After training, the saved best model is used for an independent evaluation.

The evaluation uses a deterministic policy and runs 50 episodes. The evaluation random seed is set to 20000, and the results are saved as a JSON file.

The full evaluation command is:

```
python scripts/part0/evaluate.py \  
--model checkpoints/part0/ppo_reacher_seed_0/best/best_model.zip \  
--output results/part0/ppo_reacher_seed_0/final_evaluation.json \  
--episodes 50 \  
--seed 20000 \  
--device cuda
```

The evaluation results are saved at:

```
results/part0/ppo_reacher_seed_0/final_evaluation.json
```

A summary of the 50-episode evaluation is as follows:

Metric	Value
Number of episodes	50
Mean return	-6.35 $\pm$ 3.01
Best return	-1.30
Worst return	-13.74
Mean episode length	50.0
Policy	Deterministic
Evaluation base seed	20000

The evaluation uses the default dense reward of **Reacher-v5** (distance penalty + action control penalty), with each episode fixed at 50 steps. The mean return over 50 episodes is -6.35 with a standard deviation of 3.01, indicating that the trained policy performs reasonably close to the target across different goal positions, and that the independent-evaluation part of the toolchain works correctly.

0.4 Learning Curve

The periodic evaluation data saved during training is located at:

```
results/part0/ppo_reacher_seed_0/eval/evaluations.npz
```

The following command generates the learning curve and the corresponding CSV data:

```
python scripts/part0/plot_curve.py \  
  --evaluations results/part0/ppo_reacher_seed_0/eval/evaluations.npz \  
  --output results/part0/ppo_reacher_seed_0/learning_curve.png \  
  --csv-output results/part0/ppo_reacher_seed_0/learning_curve.csv
```

The resulting learning curve is shown below:

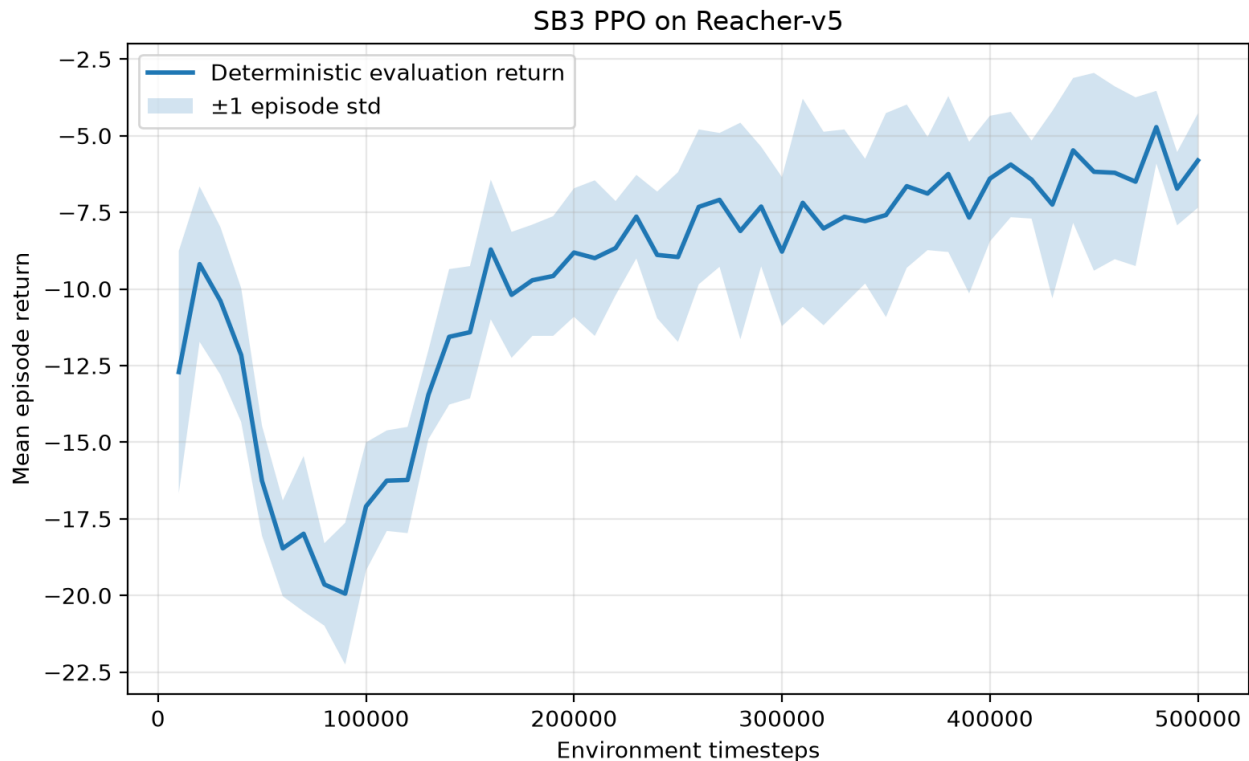


Figure 1: Learning curve of PPO on the Reacher-v5 environment (deterministic evaluation return; shaded area is ± 1 episode standard deviation)

The curve shows that the evaluation return oscillates briefly early in training (dropping to around -20 at roughly 60,000–90,000 steps), then rises steadily, converging after about 200,000 steps and stabilizing around -6, consistent with the mean return of about -6.35 over 50 episodes reported in Section 0.3.

Online view of the learning curve: https://drive.google.com/file/d/1jC9pKa9SaRUfRxjvXsLovFi2vTFLtHdk/view?usp=drive_link

0.5 Rollout Video

A rollout video is recorded using the best model saved during training. The video uses an independent random seed 30000 and is saved at 50 frames per second.

The full video-recording command is:

```
python scripts/part0/record_video.py \  
  --model checkpoints/part0/ppo_reacher_seed_0/best/best_model.zip \  
  --output videos/part0/ppo_reacher_seed_0.mp4 \  
  --seed 30000 \  
  --device cuda \  
  --fps 50
```

The generated video is located at:

```
videos/part0/ppo_reacher_seed_0.mp4
```

Online view of the rollout video: https://drive.google.com/file/d/1gsa6kyfcUfikNNX4PP49S1Vj4QAe5Fd_/view?usp=drive_link

0.6 Toolchain Verification Result

From the final rollout video, it can be directly observed that the trained PPO agent effectively controls the two-link arm, moving the end-effector to reach the given target position in the environment and completing the **Reacher-v5** reaching task. In the 50-episode independent evaluation, the policy achieves a reasonable mean return (about -6.35), which quantitatively supports the above observation.

Therefore, the following toolchain components used in these experiments all work correctly:

- Conda environment and dependency management;
- Gymnasium environment interface;
- MuJoCo physics simulation;
- Stable-Baselines3 PPO training;
- GPU model training and inference;
- checkpoint saving and loading;
- independent policy evaluation;
- learning-curve generation;
- rollout video recording.

This part is only intended to verify that the experimental toolchain works correctly, so no further analysis of PPO's performance or training process is provided.

Part 1: Custom Environment NoisyGoalReacher

1.1 Base Environment and Implementation

NoisyGoalReacher is based on Gymnasium's Reacher-v5 environment and is implemented by subclassing ReacherEnv.

Reacher-v5 was chosen because it is structurally simple and cheap to train, and because Part 0 already verified that the MuJoCo and Stable-Baselines3 toolchain works correctly.

The custom environment mainly overrides the following methods:

- `_sample_goal()`: implements goal-position randomization;
- `reset_model()`: sets a new goal at the beginning of each episode;
- `_get_obs()`: adds sensor noise to the end-effector position;
- `_is_success()`: defines the success condition;
- `_get_rew()`: implements the dense and sparse rewards;
- `step()`: returns `is_success` in `info`.

1.2 Goal-Position Randomization

The original Reacher-v5 samples the goal within a circular region around the arm base. This environment changes the goal-sampling region to an annulus:

$$r_{\min} \leq \sqrt{x_g^2 + y_g^2} \leq r_{\max}$$

The default parameters are:

```
goal_radius_min = 0.05 m
goal_radius_max = 0.18 m
```

Each time `reset()` is called, the environment resamples the goal position. First, the direction angle is sampled uniformly:

$$\theta \sim U(-\pi, \pi)$$

To make goal points uniformly distributed over the annulus area, the squared radius is sampled uniformly:

$$r^2 \sim U(r_{\min}^2, r_{\max}^2)$$

The goal coordinates are then computed:

$$x_g = r \cos \theta, \quad y_g = r \sin \theta$$

The sampled goal coordinates are written into the goal joint positions of the MuJoCo state. The goal's x and y coordinates are provided to the agent through `qpos[2:]` in the observation.

1.3 Observation Noise

Sensor noise is added only to the observation received by the agent; it does not modify MuJoCo's true physical state, nor is any noise added to the actions.

The end-effector position is obtained from the center-of-mass coordinates of the `fingertip` body in MuJoCo:

```
clean_fingertip_pos = self.get_body_com("fingertip")[:2]
```

Zero-mean Gaussian noise is then added to the 2D end-effector position:

$$\tilde{p}_{ee} = p_{ee} + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2 I)$$

The default noise standard deviation is:

```
noise_sigma = 0.005 m
```

This parameter can be configured when creating the environment. The goal position remains noise-free, and the agent receives the noisy relative position between the end-effector and the goal:

$$\tilde{p}_{ee} - p_g$$

The final observation has dimension 10, consisting of:

1. cosines of the two joint angles;
2. sines of the two joint angles;
3. the goal's x and y coordinates;
4. the angular velocities of the two joints;
5. the noisy 2D relative position between the end-effector and the goal.

1.4 Success Determination and Reward Function

Whether the task succeeds is determined by the Euclidean distance between the true end-effector position and the true goal position:

$$d = \|p_{ee} - p_g\|_2$$

When the distance is smaller than the success threshold, the task is deemed successful:

$$\text{is_success} = (d < d_{\text{success}})$$

The default success threshold is:

```
success_threshold = 0.04 m
```

Success determination uses the noise-free true positions, so sensor noise does not directly affect the environment's internal success label. Each time `step()` is called, the environment returns the success status through the following field:

```
info["is_success"]
```

The environment supports both dense and sparse reward modes.

Sparse Reward

The sparse reward uses the 0/-1 form required by the task:

$$r_{\text{sparse}} = \begin{cases} 0, & d < d_{\text{success}} \\ -1, & \text{otherwise} \end{cases}$$

The reward is 0 on success and -1 otherwise. The sparse reward contains no action penalty. The corresponding code is:

```
if self.reward_type == "sparse":
    is_success = distance < self.success_threshold
    reward_dist = 0.0 if is_success else -1.0
    reward_ctrl = 0.0
```

Dense Reward

The dense reward consists of a distance penalty and an action control penalty:

$$r_{\text{dense}} = -w_d d - w_c \|a\|_2^2$$

The distance term encourages the end-effector to gradually approach the goal, and the action control term suppresses excessively large control inputs, producing smoother and more stable motion.

The code uses the distance reward weight and action control weight inherited from `Reacher-v5`:

```
reward_dist = -distance * self._reward_dist_weight
reward_ctrl = -np.square(action).sum() * self._reward_control_weight
```

The reward is computed using the true end-effector and goal positions, not the noisy observation. This ensures that sensor noise only affects the policy input and does not change the environment's true task objective.

Reward Design Rationale

The design rationale for the two reward modes is as follows.

The sparse reward uses the standard 0 / -1 form, which is both the conventional definition of a sparse reward and a task requirement. It only distinguishes “whether the goal is reached” and contains no shaping term (such as an action penalty), so it provides the sparsest signal but is perfectly aligned with the task objective.

The dense reward is designed to provide a continuous gradient signal throughout exploration: the distance penalty $-w_d \cdot d$ makes the penalty smaller as the end-effector gets

closer to the goal, thereby guiding the policy toward the goal in any state and alleviating the “almost no feedback early in exploration” problem of the sparse reward; the action control penalty $-w_c \cdot ||a||^2$ is inherited from `Reacher-v5` and is used to suppress excessively large control inputs and encourage smoother, more energy-efficient motion. The weights of both terms directly inherit the default values of `Reacher-v5` to remain consistent with the standard Reacher task.

It should be noted that this initial dense reward only contains “distance penalty + control penalty” and does not provide any additional incentive for the event of “crossing the success threshold” itself. This design trade-off exposed a problem in the experiments of Part 3, and is analyzed and improved as a failure mode in Part 4.

1.5 Environment Check and Unit Tests

The custom environment has been verified using Gymnasium’s environment checker `gymnasium.utils.env_checker.check_env`, with the result being a pass (`Gymnasium environment check passed`), indicating that the observation, reward, termination flags, and `info` returned by the environment satisfy the basic requirements of the Gymnasium API. The actual runtime output is:

```
$ python scripts/part1/check_custom_env.py
.../gymnasium/utils/env_checker.py:317: UserWarning: WARN: A Box observation
space minimum value is -infinity. This is probably too low.
.../gymnasium/utils/env_checker.py:321: UserWarning: WARN: A Box observation
space maximum value is infinity. This is probably too high.
Gymnasium environment check passed.
```

The two `UserWarnings` emitted during the check indicate that the observation space bounds are $\pm\text{inf}$. This is expected behavior: the observation of this environment includes quantities without clear physical bounds, such as joint angular velocities and the noisy relative position, so it follows the `Box(-inf, inf, ...)` definition commonly used in Gymnasium (consistent with the original `Reacher-v5`). These are informational warnings and do not affect passing the check.

In addition, the following unit tests are written using `pytest`:

1. Goal-region test.

Reset the environment consecutively with multiple random seeds and verify that the distance between the sampled goal and the origin always lies within the specified annulus range:

```
0.05 m <= goal radius <= 0.18 m
```

2. Observation-spec test.

Verify that the observation returned by the environment has dimension `(10,)`, and confirm that the observation belongs to the environment’s declared `observation_space`.

3. Sparse-reward test.

Verify that the sparse reward can only take `-1` or `0`, and check that the reward value is consistent with `info["is_success"]`.

4. Success-flag test.

Execute multiple environment steps consecutively and verify that each call to `step()` always includes a boolean `is_success` field in `info`.

All tests can be run with the following command:

```
pytest
```

The actual run result is as follows; all 4 tests pass:

```
$ pytest
===== test session starts =====
platform linux -- Python 3.11.15, pytest-8.4.1, pluggy-1.6.0
rootdir: /home/movel/rl-robotics-takehome
configfile: pyproject.toml
testpaths: tests
plugins: typeguard-4.5.2
collected 4 items

tests/part1/test_noisy_goal_reacher.py .... [100%]

===== 4 passed in 0.18s =====
```

Part 2: Custom PPO Implementation vs. SB3 Baseline

2.1 Implementation

This part implements and trains a custom PPO (Proximal Policy Optimization) agent on the custom environment `NoisyGoalReacher-v0`. The custom PPO is adapted from CleanRL's single-file continuous-action PPO (`ppo_continuous_action.py`) (source: <https://github.com/vwxyzjn/cleanrl>), where the Generalized Advantage Estimation (GAE) part is implemented from scratch for on-policy methods and marked in the code with a `# implemented from scratch` comment. As a baseline, the standard PPO algorithm from Stable-Baselines3 (SB3) is used for comparison under exactly the same environment, reward, training budget, and evaluation protocol.

PPO was chosen as the algorithm for the following reasons: PPO is relatively simple to implement, and by clipping the objective function it limits the size of each policy update, making training relatively stable and not overly sensitive to hyperparameters; it is a mature and widely used on-policy baseline algorithm for continuous-control tasks, suitable for a mid-scale continuous-control problem like this one; furthermore, SB3 provides a well-tested PPO implementation, making it convenient to use as a baseline for a fair comparison against the custom implementation.

The training scripts for the custom implementation and the SB3 baseline are located at:

```
scripts/part2/train_custom_ppo.py    # custom PPO
scripts/part2/train_sb3.py           # SB3 baseline
```

2.2 Training Configuration

The two implementations use aligned key hyperparameters (learning rate, per-update batch size, discount factor, GAE λ , clip coefficient, etc.) to ensure a fair comparison. The batch size collected per policy update is 2048 for both implementations. The environment and training parameters are:

Parameter	Value
Environment ID	NoisyGoalReacher-v0
Goal radius min / max	0.05 m / 0.18 m
Observation noise σ	0.005 m
Success threshold	0.04 m
Reward type	Dense
Random seed	1
Total timesteps	1,000,000 (actual 1,001,472)
Batch size (per update)	2048
Minibatch size	64
Update epochs	10
Learning rate	3e-4
Discount factor γ	0.99
GAE λ	0.95
Clip coefficient ϵ	0.2
Entropy coefficient (ent_coef)	0.0
Value function coefficient (vf_coef)	0.5

Parameter	Value
Gradient clipping (max_grad_norm)	0.5
Device	auto

2.3 Comparison with the SB3 Baseline

Under the same environment, the same reward (dense), the same training budget (1M steps), and the same evaluation protocol, the custom PPO is compared against SB3 PPO (both seed 1). The top subplot is the mean episode return (shaded area is ± 1 standard deviation), and the bottom subplot is the success rate as a function of training steps.

Analysis. In terms of mean episode return, the custom PPO (orange) stays essentially above the SB3 baseline (blue) throughout training, rising more smoothly and finally converging to about -3.7, while SB3 converges to the range of about -4 to -5. In terms of success rate, the two are intertwined during the early-to-mid phase with little difference; after about 0.6M steps, the custom PPO gradually pulls ahead, finally reaching a success rate of about 0.9 versus about 0.7 for the SB3 baseline. Overall, under this seed, the custom PPO is slightly better than the SB3 baseline on both metrics, with the success-rate advantage being more pronounced later in training.

It should be noted that the above comparison is based on a **single random seed (seed 1)**, so the gap between the two implementations may partly stem from randomness and is not sufficient to claim that the custom PPO is statistically superior to the SB3 baseline. The main conclusion here is that the custom PPO can train and converge normally on this environment, and its performance is at the same level as the standard SB3 PPO (and slightly better under this seed). A more rigorous comparison would require repeating the experiment over multiple seeds and reporting the mean and variance.

2.4 Reproducible Command

An example training command for the custom PPO:

```
python scripts/part2/train_custom_ppo.py --seed 1 --reward-type dense
```

The remaining full commands (including the SB3 baseline and different configurations) are in:

```
configs/part2_commands.txt
```

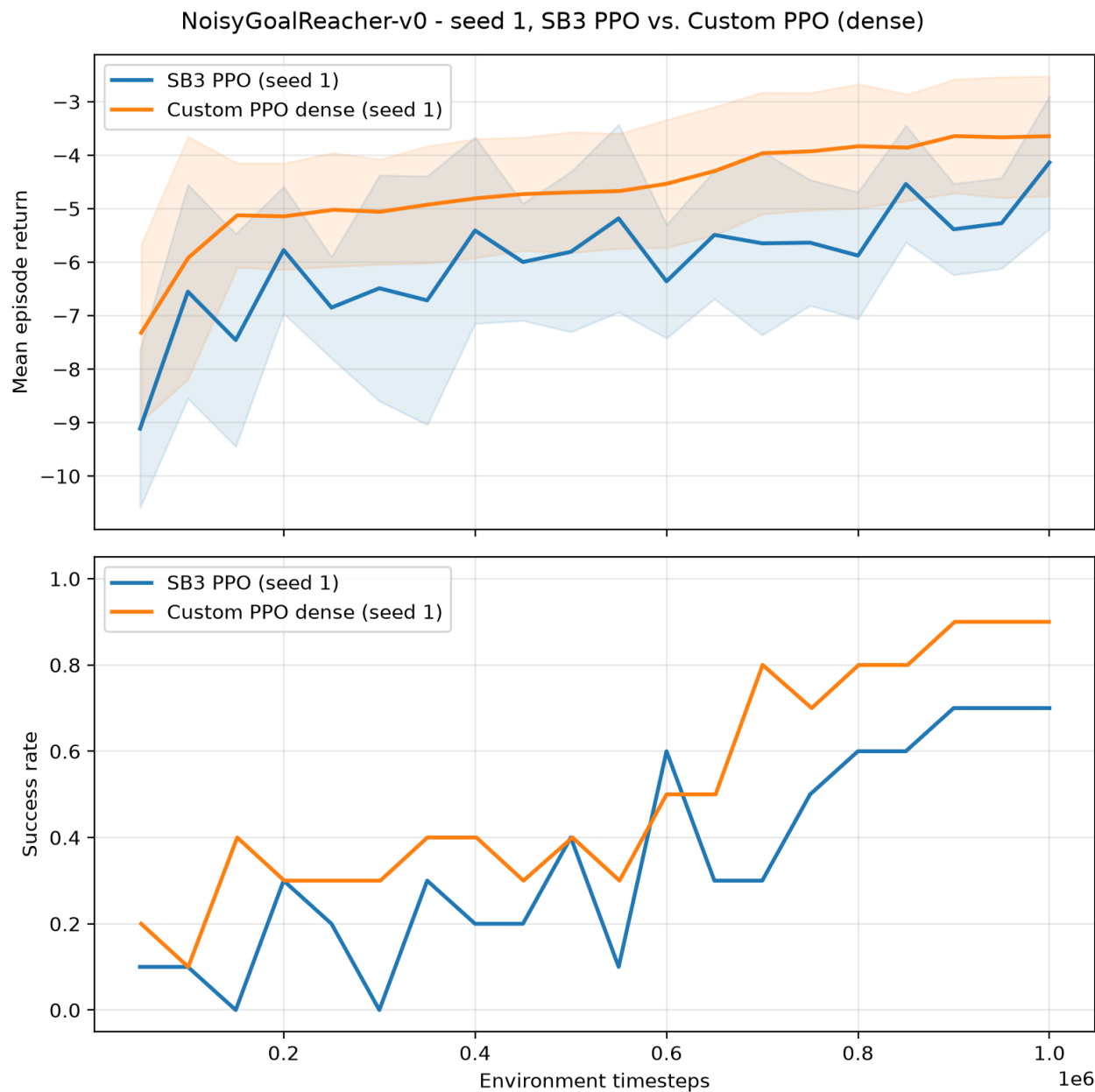


Figure 2: Comparison of SB3 PPO and Custom PPO (dense) on NoisyGoalReacher-v0 (seed 1): top is mean episode return, bottom is success rate

Part 3: Reward and Robustness Study

This part completes two small experiments: a comparison between sparse and dense rewards, and a brief explanation of HER (Hindsight Experience Replay). Each configuration uses 3 random seeds.

3.1 Sparse Reward vs. Dense Reward

Under the same training budget (1M steps), the custom PPO is trained with the sparse reward and the dense reward respectively, averaged over 3 random seeds. The figure below shows the success rate of the two reward modes as a function of training steps (i.e., the success-rate learning curve). Here the success rate is presented as the learning curve; the mean episode return is not plotted as a separate curve because the two rewards have different scales (see the analysis below) and are not directly comparable.

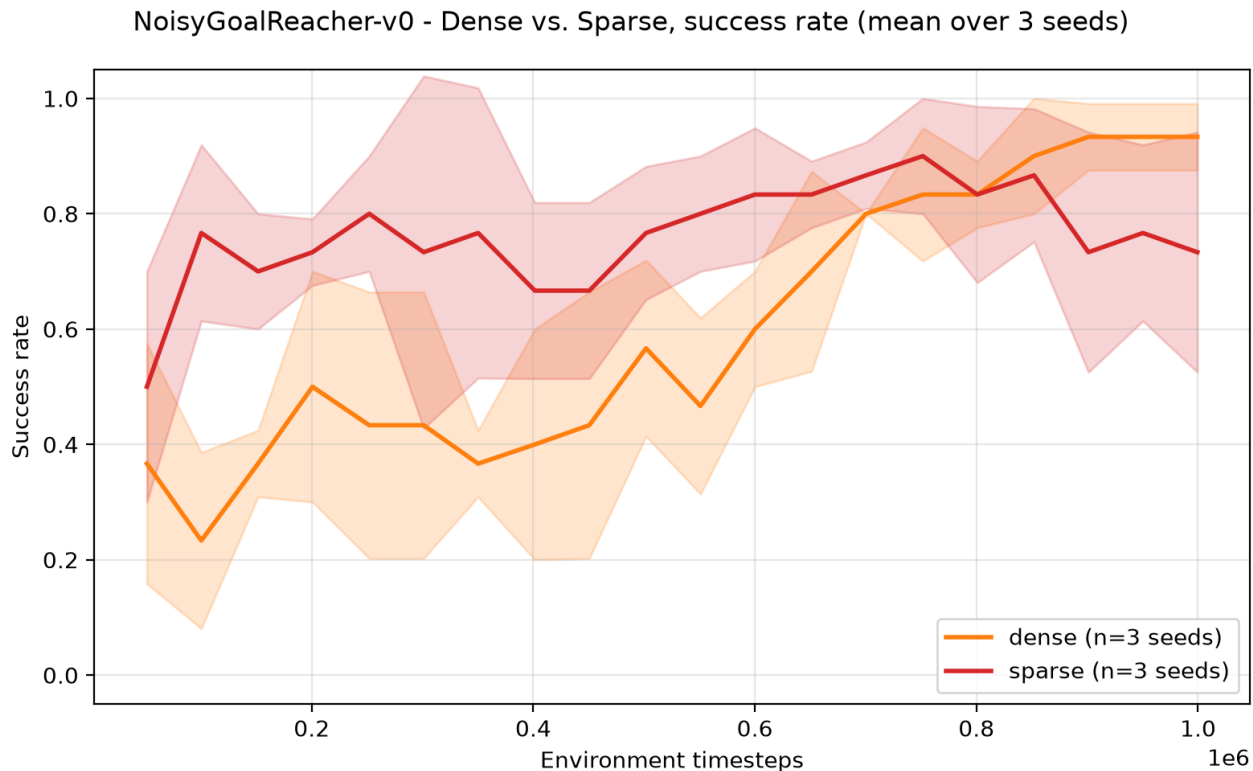


Figure 3: Comparison of success rate for dense vs. sparse rewards on NoisyGoalReacher-v0 (mean over 3 seeds; shaded area is the standard deviation across seeds)

Comparative Analysis

The sparse-reward training **did not fail**: both reward modes completed training and learned the task under the 1M-step budget, so there is no “sparse-reward training failure.” The discussion below focuses on the differences between the two.

A counterintuitive but genuinely observed phenomenon in these experiments is: **in terms of success rate, the sparse reward is not worse than the dense reward, and is even slightly**

better in most phases. When interpreting this, the two metrics must be considered separately.

Return. The mean episode return of the dense reward is clearly higher than that of the sparse reward (dense is about -5 to -30, sparse is about -45 to -50). But this gap cannot indicate which learns better, because the two rewards are on different scales: the dense reward is negative distance plus negative action penalty, while the sparse reward is an accumulation of 0 or -1 per step, and an episode of 50 steps that never reaches the goal is about -50. Therefore, the dense reward being higher in return is inevitable and cannot be used to compare the two rewards horizontally.

Success rate. The truly comparable metric is the success rate, and the sparse reward rises faster and is slightly higher overall in success rate. The sparse reward only gives a reward when the end-effector enters the success threshold and provides no effective feedback otherwise, so exploration yields little information; by general reasoning it should be harder to train. But on this task, the actual result is the opposite of that general rule.

The reason is that the sparse reward’s optimization objective is directly aligned with the success-rate metric used for evaluation: the sparse reward itself is “whether the goal is reached” (0 / -1), and what it optimizes is the success rate. The initial dense reward, however, optimizes a weighted objective of “distance plus control penalty” and provides no additional incentive for the event of “crossing the success threshold and truly reaching the goal.” Therefore, when the end-effector is already close to the goal but has not yet crossed the success threshold, the return under the dense reward is already high (the distance term is small), so the motivation to make a final push into the success circle is relatively weak; the sparse reward, on the other hand, only distinguishes success from failure and keeps pushing toward the threshold. This design flaw of the dense reward and its improvement are analyzed and verified further as a failure mode in Part 4.

This also reflects a typical trade-off of reward shaping: the dense reward provides a denser learning signal and is theoretically easier to learn, but if the shaped reward is not fully aligned with the true objective of “completing the task” (for example, only penalizing distance without incentivizing “reaching” itself), its optimization direction may deviate from the true objective; the sparse reward has a sparse signal but is perfectly aligned with the true objective. The success threshold of 0.04 m in this task is not too strict, and the sparse signal is not so sparse as to be unlearnable, so the advantage of objective alignment shows up.

3.2 HER (Hindsight Experience Replay)

What is HER

HER (Hindsight Experience Replay) is a method designed for sparse rewards, commonly used in goal-conditioned difficult tasks in continuous action spaces, such as robotic-arm or robot grasping. Its core idea is to **turn failed experience into successful experience for reuse**.

The implementation mechanism is as follows: for a collected transition, if the agent did not reach its originally set goal, then under the original goal this experience earns almost no reward (under the sparse reward it stays at -1, a failed experience). HER copies this transition, replaces the goal with the position the agent actually reached at that time, and recomputes the reward under this hindsight-relabeled goal, turning the experience into a “successfully

reaching the goal” experience. Experience that was originally a failure and lacked learning value thus becomes an effective positive-feedback sample.

Why HER may help solve the sparse-reward problem

The main difficulty of sparse rewards is that early in exploration the agent can almost never reach the goal, so the vast majority of experiences are “not reached, reward constantly -1” failures, providing very little effective learning signal and making training hard to get started.

HER alleviates this: even if the agent does not reach the original goal, it always reaches some position; after relabeling the goal to that actual position, the experience becomes a meaningful “how to reach that position” successful sample. As a result, even if the original reward is extremely sparse, the agent can learn from every trajectory how to reach a given position, significantly improving sample efficiency and making sparse-reward tasks learnable.

3.3 Noise Sensitivity

Training and Evaluation Setup

This experiment uses the best-performing dense-reward policy for noise-sensitivity evaluation. Among the three seeds in Part 2, the dense-reward policy of seed 1 had the highest success rate at the training noise level ($\sigma = 0.005$), at 92%, better than seed 12 (88%) and seed 1300 (70%); therefore the dense-reward policy of seed 1 was selected as the evaluation target.

The approach used is **train at a fixed σ , evaluate at other σ values**: after this policy was trained at training noise $\sigma = 0.005$, it was evaluated directly at three noise levels $\sigma \in \{0, 0.005, 0.02\}$, without randomizing σ during training. Each noise level runs 50 deterministic episodes with evaluation random seed 3000, and the results are saved respectively at:

```
results/part2/final_eval/dense_seed1_noise_sigma_0.json
results/part2/final_eval/dense_seed1_noise_sigma_0.005.json
results/part2/final_eval/dense_seed1_noise_sigma_0.02.json
```

The corresponding evaluation commands are in the new “Sensor-noise sweep” section of `configs/part2_commands.txt`.

Results

Evaluation noise σ	Relative to training noise	Success rate	Mean episode return
0	$0\times$ (no noise)	92%	≈ -4.23
0.005	$1\times$ (training level)	92%	≈ -4.23
0.02	$4\times$	92%	≈ -4.24

Over the entire range of σ from 0 to 0.02, the success rate stays stable at 92%, and the mean episode return is essentially unchanged (about -4.23, dropping only slightly to about -4.24 at

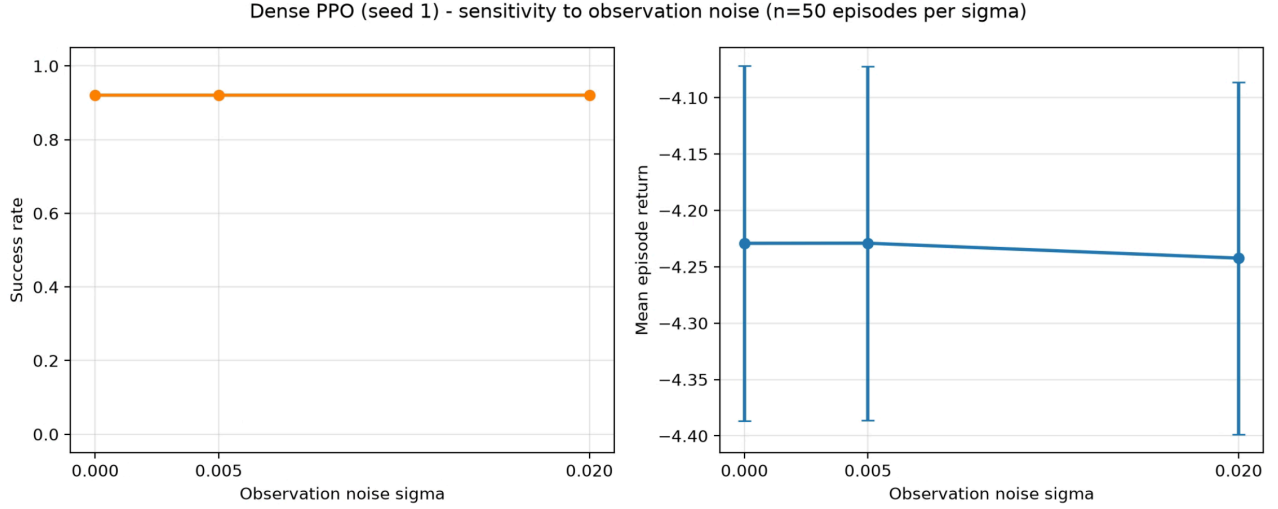


Figure 4: Sensitivity of the Dense PPO (seed 1) policy to observation noise: left is success rate, right is mean episode return (50 episodes per σ ; error bars are standard deviation)

$\sigma = 0.02$, with the error bars of the three noise levels overlapping). Even when the evaluation noise reaches $4\times$ the training noise, the policy performance shows no obvious degradation. This indicates that the dense-reward policy is fairly robust to observation noise: within up to $4\times$ the training noise level, noise does not significantly affect its ability to reach the goal.

A reasonable explanation is that the observation noise only affects the end-effector-position part of the observation, its standard deviation (≤ 0.02 m) is still small relative to the workspace scale and the success threshold (0.04 m), and the policy has already been exposed to $\sigma = 0.005$ noise during training, so it is insensitive to moderate amplification of the same type of noise.

Applying to a Real Robot

To transfer this approach to a real robot, I would **randomize σ during training** (i.e., domain randomization) rather than train at a fixed σ . The reason is that real sensor noise is usually not a stable quantity; its magnitude varies with hardware, environment, and operating conditions, and may be higher or lower than a fixed value. A policy trained at a fixed σ has no guarantee of generalizing beyond the training noise level, whereas randomizing over a range of noise during training lets the policy stay stable over a wider range of noise levels.

One caveat should be added: the noise in this experiment is added as zero-mean Gaussian noise, whereas real sensor noise does not necessarily follow a Gaussian distribution (there may be bias, quantization, outliers, etc.), so randomizing only the standard deviation of Gaussian noise cannot fully cover the characteristics of real noise. Nevertheless, when conditions allow, randomizing σ during training is still a more robust choice than a fixed σ .

Part 4: Evaluation Protocol, Failure Modes, and Future Work

4.1 Evaluation Protocol

The evaluation of each experiment in this report follows a unified protocol, with the evaluation code separated from the training scripts and placed in a standalone evaluation script (`scripts/part0/evaluate.py` and the corresponding evaluation script for Part 2). The key points of the protocol are:

- **Deterministic policy.** Deterministic actions are used during evaluation (the continuous policy takes the mean of the Gaussian distribution without sampling), to remove the effect of action randomness on the results.
- **Number of evaluation episodes.** Each configuration runs at least 50 evaluation episodes (the final evaluation in Part 0 is 50 episodes, and the noise-sensitivity evaluation in Part 3 is also 50 episodes per σ). The periodic evaluation during training is 10 episodes each, used only for plotting learning curves.
- **Recording random seeds.** Every evaluation records and fixes a random seed (e.g., seed 20000 for the final evaluation in Part 0, seed 30000 for video recording, seed 3000 for the noise-sensitivity evaluation), ensuring reproducible evaluation conditions and comparability across models.
- **Reported metrics and dispersion.** The success rate and mean episode return are reported, together with a measure of dispersion (return standard deviation; Part 0 gives mean $\pm$ std, and the comparison figures in Part 3 use shaded areas to indicate the standard deviation across seeds).

4.2 Failure Modes and Debugging

Dense-Reward Design Flaw: Missing Arrival Reward

The main problem in this project appeared in the design of the dense reward and was only exposed through the comparison experiments in Part 3.

Phenomenon. In the dense-vs-sparse comparison of Part 3, a counterintuitive result appeared: in terms of success rate, the dense reward was not better than the sparse reward and was even slightly lower than it in most training phases. By common understanding, the dense reward provides a denser learning signal and should be easier to learn, so this result initially made me suspect an error in the implementation or the pipeline.

Localization process. I checked several possibilities in turn:

- I first suspected a plotting or data error, e.g., the two curves having swapped colors/legends. So I went back to the original per-seed curves and checked them one by one (the dense-vs-sparse dual plots for seed 1 / 12 / 1300), confirming that the legend and data corresponded correctly and that the sparse reward having a higher success rate than the dense reward is a reproducible real result, not a plotting error.
- I then checked the return metric and found that the return of the dense reward (about -4 to -30) is indeed much higher than that of the sparse reward (about -45 to -50). But after further analysis I confirmed that this is only due to the two rewards being on different scales (dense is negative distance plus negative action penalty, sparse is an accumulation of 0/-1 per step); the return cannot be used for horizontal comparison and does not indicate that dense learns better. On the truly comparable success-rate

metric, dense does not have the advantage.

- After ruling out plotting errors and metric misreading, I localized the problem to the design of the reward function itself.

Root cause. The initial dense reward is $-w_d \cdot d - w_c \cdot ||a||^2$, containing only a distance penalty and a control penalty, with **no additional incentive for the event of “crossing the success threshold and truly reaching the goal.”** That is, the reward the end-effector receives just outside the success threshold (e.g., 0.041 m) and just inside it (0.039 m) is almost the same, so the policy cannot perceive the difference between “reached” and “close but not reached,” and thus lacks the motivation to make a final precise push. In contrast, the sparse reward’s 0/-1 signal is directly aligned with the success-rate metric, so it performs better on success rate.

Fix. To address this flaw, I added an arrival reward (success bonus) to the dense reward: when the end-effector distance is smaller than the success threshold, an additional positive reward is given, so that the event of “reaching the goal” itself has a clear positive incentive. The modified dense reward is:

```
r_dense = -w_d · d - w_c · ||a||2 + (success_bonus if d < d_success, else 0)
```

where `success_bonus` is set to 3.0. To keep attribution clean, this modification only adds the arrival reward and keeps everything else (distance weight, control penalty weight) unchanged, forming a single-variable comparison of “original dense reward vs. dense reward with the arrival reward added.”

Verification result. Over 3 random seeds, the original dense reward, the new dense reward with the arrival reward added, and the sparse reward were compared on success rate (in the figure below, `dense` is the original dense reward, `dense_n_rew` is the new dense reward with the arrival reward added, and `sparse` is the sparse reward; all are means over 3 seeds, with shaded areas being the standard deviation across seeds).

The result confirms the diagnosis above: the new dense reward with the arrival reward added (`dense_n_rew`) has a higher success rate than the original dense reward (`dense`) throughout training, with the improvement especially pronounced in the early-to-mid phase; by late training, the new dense reward converges to about 0.97, the highest and most stable of the three, surpassing the sparse reward (`sparse` falls back to about 0.73 late in training). This shows that the disadvantage of the original dense reward on success rate indeed mainly comes from “the missing arrival reward,” and after adding this term, the dense reward not only fixes the original deficiency but also ends up outperforming the sparse reward.

Other Debugging Items

- **Early-training return oscillation.** On the learning curve in Part 0, PPO’s evaluation return dropped noticeably at around 60,000–90,000 steps (down to about -20 at one point) before rising and converging. This was judged to be the policy oscillation commonly seen early in on-policy training rather than an implementation error, as confirmed by the stable subsequent rise of the training curve.
- **Duplicate assignment in the reward function.** During code review, a duplicate `reward = reward_dist + reward_ctrl` assignment (overwritten by the immediately following line) was found in `_get_rew`; although it does not affect the computed result, it was

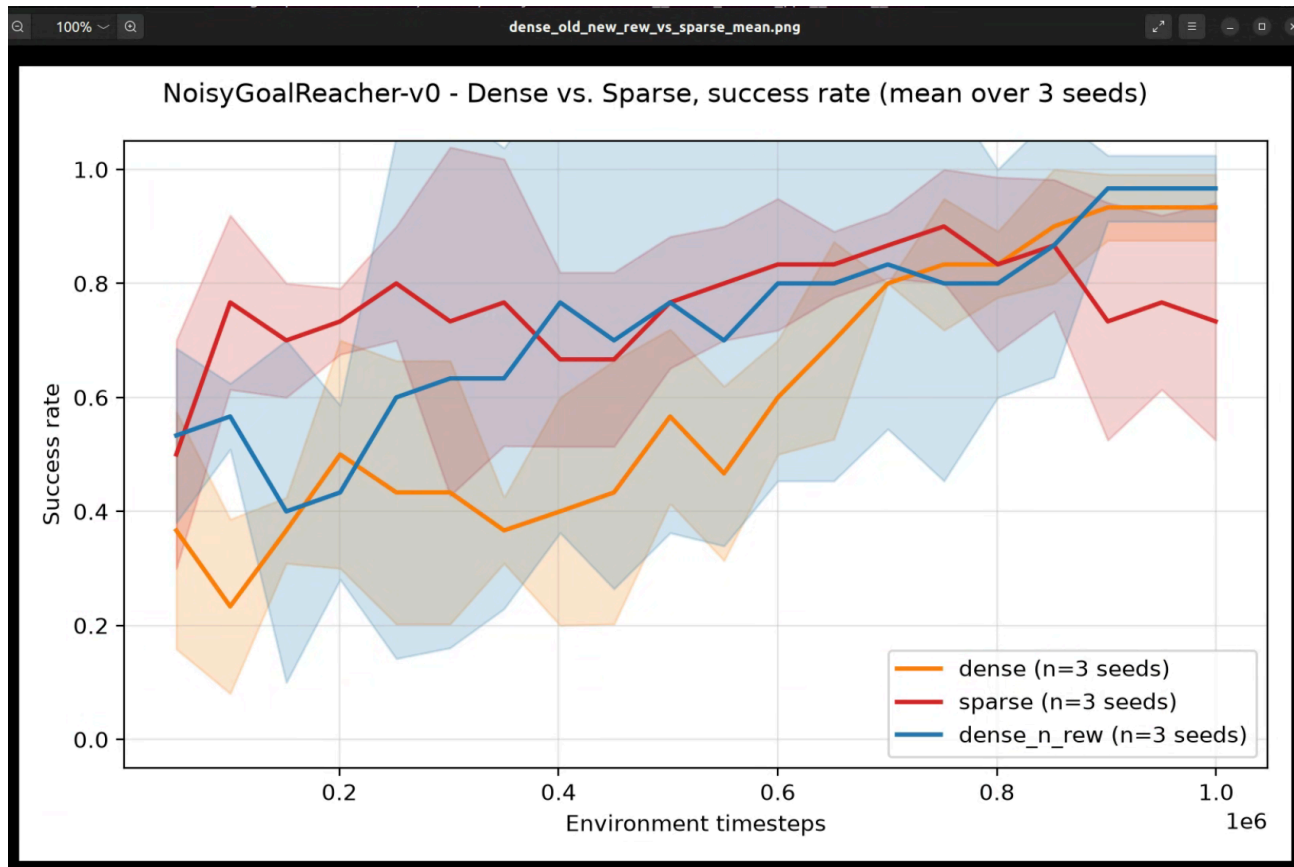


Figure 5: Comparison of success rate for the original dense reward, the new dense reward with arrival reward, and the sparse reward on NoisyGoalReacher-v0 (mean over 3 seeds; shaded area is the standard deviation across seeds)

cleaned up while tidying the reward function.

Compatibility Issues in Environment and Dependency Integration

In the process of integrating CleanRL's PPO implementation with the current version of Gymnasium and the SB3 baseline into the custom environment, several library-version compatibility issues were encountered. Some manifested as direct runtime errors, and others were migration changes made to adapt to the new Gymnasium API.

Errors encountered and fixed:

- **SB3 requires the reward to be a Python float.** When validating environment return values, SB3 requires the reward to be a scalar `float`, whereas the reward computed by NumPy operations in `_get_rew` defaults to a NumPy scalar (e.g., `np.float64`); returning it directly triggers a type error. After localizing this issue, the reward is explicitly converted to a Python `float` with `float(...)` before being returned (i.e., `reward = float(reward_dist + reward_ctrl + reward_success_bonus)`), and the error is eliminated.
- **Change in how episode statistics are read.** The new version of Gymnasium's `RecordEpisodeStatistics` changed the structure in which episode statistics are stored in vector environments: the old version placed each environment's statistics in per-environment dicts in `final_info` (a list, read per env via `info["episode"]["r"]`), while the new version changed to a vectorized structure, placing the statistics in the `infos["episode"]` array with an `infos["_episode"]` boolean mask marking which environments finished at the current step. The old per-dict reading approach errors under the new structure, and it was fixed by reading in bulk via the `infos["episode"]` array plus the `_episode` mask (`np.where(mask)`).

Migration changes made to adapt to the new Gymnasium:

- **Splitting done into terminated / truncated.** The old CleanRL's `env.step()` returns a 4-tuple (`obs, reward, done, info`), while the new Gymnasium returns a 5-tuple (`obs, reward, terminated, truncated, info`). Beyond the unpacking, more importantly the two must be distinguished semantically in the GAE computation: on true termination (`terminated`), the future value is not bootstrapped (`delta = reward - value`), whereas on time truncation (`truncated`), it must be bootstrapped (`delta = reward +  $\gamma$  \cdot value_{next} - value`). The old single `done` cannot distinguish these two cases and would incorrectly zero out the future value at time truncation.
- **Explicitly setting the autoreset mode.** The new Gymnasium makes the auto-reset behavior of vector environments a configurable mode, and the vector environment is constructed with an explicit `autoreset_mode=AutoresetMode.SAME_STEP` to match the rollout and bootstrap logic used.
- **Using the true final observation for value bootstrap at truncation.** Under the autoreset semantics, the `next_obs` returned by the environment after an episode ends is already the initial observation of the new episode; therefore, at time truncation the bootstrap value cannot be computed directly from `next_obs`, but must be computed from the true final observation taken from `infos["final_obs"]`, and this logic is handled separately in the rollout loop.

4.3 Future Work

With more time, I would try the following two directions. Each targets a specific, already-observed limitation of this project:

- **Introducing SAC + HER for sparse rewards.** Part 3 shows that the sparse reward, while learnable, provides almost no effective signal early in exploration (the reward of the vast majority of experiences is constantly -1). HER (Hindsight Experience Replay) can relabel failed experiences that did not reach the goal into successful experiences according to the actually reached position, significantly improving sample efficiency. But HER relies on a replay buffer to repeatedly reuse the relabeled experiences, which is incompatible with the current on-policy PPO; therefore, one would need to switch to the off-policy SAC as the base algorithm to use HER.
- **Randomizing the observation noise σ during training (domain randomization).** The noise-sensitivity experiment in Part 3 trains at a fixed σ . Real sensor noise levels are usually unknown and fluctuate, and a policy trained at a fixed σ has no guarantee against noise beyond the training level; randomizing over a range of σ during training can improve the policy's robustness to a wider range of noise levels, which is more favorable for transfer to a real robot.